Learning Objectives

- Testing in SDLC
- Difference between Testing and Debugging
- Explain the different types of testing methods
- Develop a plan for a software test

What is Software Testing?

Software Development Life Cycle (SDLC)

Video Resource of SDLC

https://youtu.be/Fi3_BjVzpqk?si=z1eJm1mSm_2AbgtT

Software testing

- Software testing is the process of checking that all the features and functions of a program work as they should do and give the correct results to the tests they are subjected to. What is meant by terms such as ‘**work as they should**’ and ‘**correct results**’ needs to come from the original **program specification that was agreed by the client/users and the software developers** before the program was written.

1)In Pairs, Test it, then debug the code to fix the errors.

- What is the difference between testing and debugging?

```
def add_numbers(a, b):
    return a - b
x = input("Enter first number: ")
y = input("Enter second number: ")

result = add_numbers(x, y)
print("The sum is:", result)
```

Testing Vs Debugging

Aspect	Testing	Debugging
Purpose	Identifies bugs and verifies requirements.	Finds and fixes the root cause of bugs.
Process	Involves running the software in various scenarios.	Involves analysing and correcting the code.
Who Performs It	Testers or QA engineers.	Developers.
Automation	Can be automated with tools and scripts.	Mostly manual, supported by debugging tools.
Goal	Ensures software quality and correctness.	Fixes defects for proper functionality.

Test and Debug example

1. **Test the Program**:
   - To identify the issues.
   - Expected errors:
     - The program will incorrectly subtract instead of adding.
     - Input from input() is treated as a string, so passing numbers will lead to unexpected results.

2. **Debug the Program**:
   - **Bug 1**: The subtraction operator should be replaced with the addition operator.
   - **Bug 2**: Convert x and y to integers or floats before passing them to the add_numbers() function.

Types of data Testing

- Functionality testing
- Stability testing
- Usability testing

Functionality Test

- Tested to ensure that it conforms to all its user **functionality requirements**. It does this by using the requirements specification that was written in the analysis stage of the software development life cycle. The functionality of a software program is the **actions and calculations that it should be able to carry out**. Another phrase for functionality testing is ‘**black box testing**’ which, just like functionality testing, **ensures that all the requirements are met**.

Example

```
def add(a, b):
return a + b
```

Test case 1: Adding two positive numbers: add(2, 3) should return 5.

Test case 2: Adding a positive and a negative number: add(5, -3) should return 2.

Test case 3: Adding two negative numbers: add(-2, -4) should return -6.

Test case 4: Adding zero and a number: add(0, 5) should return 5.

Test case 5: Adding zero and zero: add(0, 0) should return 0

Stability Test

- Stability testing is often referred to as endurance testing. This method of testing checks to see if the software program can **continuously function well for an acceptable period of time or longer**. The program will be run continuously to ensure that the program is robust and will continue to work as expected under prolonged conditions.

Example

```
def add(a, b):
    return a + b
```

Stress Testing

Test Case 1: Add two very large positive numbers.

Test Case 2: Add two very large negative numbers.

Test Case 3: Add zero and a very large number.

Test Case 4: Run the addition function 1 million times with different values to ensure stability over repeated executions.

Usability Test

- Usability testing is testing of the software program by **different users**. This is essentially an evaluation of the software and is a good opportunity to refine and improve the program, as necessary. Usability testing (or user testing as it may be called) is invaluable because it gives developers **direct feedback on how real users deal with the system**, that is, information about how well it works in general and any specific issues that the test users encountered.

Example

```
def add(a, b):
return a + b
```

Test Case 1: Valid Input (Positive Numbers)

Test Case 2: Invalid Input (Non-numeric)

Test Case 3: Mixed Valid and Invalid Input

Test Case 4: Zero as Input (0 and 0.0)

What do you test?

Type of test	What it tests	Things to remember
Functionality	Do the workings of the program match user requirements?	• Functionality testing requires no understanding of the underlying program code. • Black-box testing focuses on this type of testing.
Stability	How reliably does the program perform under difficult conditions?	• Issues such as high data volumes or excessive number of users can place stress on a programmed solution, affecting its stability. • Tests for stability often deliberately go beyond the expected performance limits of a program.
Usability	How easy is the program to operate?	• Best performed by target users who have no involvement in the development process so are unlikely to be biased. • Users are typically observed, to document the issues and confusion that they may encounter whilst using the software, or they may be interviewed or complete questionnaires.

- Provide an example on Functionality Testing, Stability Testing, and Usability Testing.

Three main categories of test data

There are three main categories of test data: normal, abnormal and extreme. Developers need to know how the program will respond to all three types of data.

- ❑Normal test data
- ❑Abnormal test data
- ❑Extreme test data

Normal Test Data

- Normal test data is data that you would normally expect as an input value for your program. For example, if your program were for a gym, people would need to input their ages on the application form for the gym and normal test data would include values such as:
  - 21
  - 40
  - 55
  - 68.

Abnormal Test Data

- Abnormal test data are input values which are incorrect entries for that part of the program. For example, when inputting ages of gym members, the following would all be incorrect because a number that could be a person's age is required:
  - 31/01/1984 – because it is a date rather than an age.
  - 250 – because, although it is a number, it is not realistic for a person's age.
  - Forty five – because it is a text value where a numerical value is expected. (Using numbers consistently would allow mathematical analysis to be carried out using the data set of gym members' ages.)

Abnormal Test Data (Cont'd)

- If a date of birth were called for instead of an age, then 31/01/1984 from the list above would be acceptable (normal test data) but 31/02/1984 would be abnormal, because there are only 28 (or 29) dates in February.

Extreme test data

- Extreme test data is data at the boundary between typical (normal) data and invalid (abnormal) data. For example:
- ‣ a textbox where someone’s name is to be entered:

extreme values might include ‘Tc’, ‘123456789’, ‘xxxxx’, or a very long name.

- ‣ a validation rule where only numbers between 10 and 100 can be entered: typical test data on the boundaries would be input values of 100, 99, 10 and 11.

Test data choices

Normal test data	Data which is commonly entered and is within normal use and expected ranges
Extreme test data	Data which is uncommon or at the edges of expected ranges but is not invalid
Abnormal test data	Data which is invalid but could be entered by a user in error

What does test data do?

- ✓ Checks inputs
- ✓ Checks data storage, especially appropriateness of selected variable data types and structures
- ✓ Checks calculations are correct
- ✓ Checks program logic including selections and iterations
- ✓ Checks that validation methods are working correctly (especially for extreme and abnormal test data)
- ✓ Checks outputs
- ✓ Checks that functionality works as required

Individual Task

- Choose one of the program written in the previous lessons and test with different input data. Record the data and the results in a test plan file.

Developing a Test Plan

Test Number	Purpose of Test	Test Data	Expected Result	Actual Result	Comments

Group Task

- Swap place with a colleague and go through the tests in the written test plan. Record additional tests.

Extension activity

Record in a test plan the normal, extreme and abnormal test data.

Now try this

As part of an educational software development project a program is required that simulates a bank's automatic teller machine (ATM), offering the user typical banking facilities such as:

1. enter PIN
2. deposit cash
3. withdraw funds
4. change personal identification number (PIN)
5. print a statement
6. exit ATM.

Anticipated inputs include:

- existing PIN
- new PIN
- menu option
- withdrawal cash amount (pounds only)
- deposit cash amount (pounds and pence).

Determine the different inputs required for the ATM simulator program.

For each input you have identified make suggestions for potential normal, extreme and abnormal test data.

Think about:

- the program's functionality
- the program's inputs
- normal test data
- extreme test data
- abnormal test data.

Thank you

Next lesson:

Software Evaluation